Learning basic mechatronics concepts using the Arduino board and MATLAB

Giampiero Campa, PhD, Technical Evangelist
MathWorks, 970 W 190 ST, Suite 530,
Torrance, CA, 90502, USA

giampiero.campa@mathworks.com

Outline

- A few challenges with mechatronics projects
- Arduino introduction, and MathWorks packages
- Analog and Digital IO, blink challenge example
- Servo, DC, and Stepper Motors, Encoders
- Simulink blocks, more details, and useful links

Sensing & data acquisition

- Distance/Range, Position/Orientation, Force/Pressure, Touch, Orientation, Motion/Speed, Environment, Optical, Chemical, Flow, Voltage/Current, sensor selection is largely application related but also related to:

- Data acquisition system (Analog and Digital input)
  - Number of sensors
  - Ranges
  - A/D conversion issues as Resolution and Sampling Rate
  - Signal Conditioning
  - Delay/Bandwidth
  - Encoders, Interrupts, SPI, I2C

Processing & programming platform

- External Microcontroller/Microprocessor Board
  - Development-Prototyping Environment
  - Low-Level Programming Language, Compiler/Assembler
  - Connection to computer (or LCD) for visualization
  - DAQ typically already there
  - Stand Alone Issues, power requirements

- Computer
  - Connection to DAQ (USB, PCI, Serial, Wireless)
  - Programming-Analysis Environment
  - OS / Drivers (Real Time ?)
  - Embedded / Small Form Factor Computer

Acting (digital & analog outputs, PWM)

- Relays
- Motors
  - DC/Steppers/Servos
  - Signal Amplification
  - Power Suppliers, Batteries
  - Forward/Reverse, H-Bridges
- Displays

Lots of (interrelated) choices

Driving Factors:

- Cost
- Easy of Use
- Flexibility

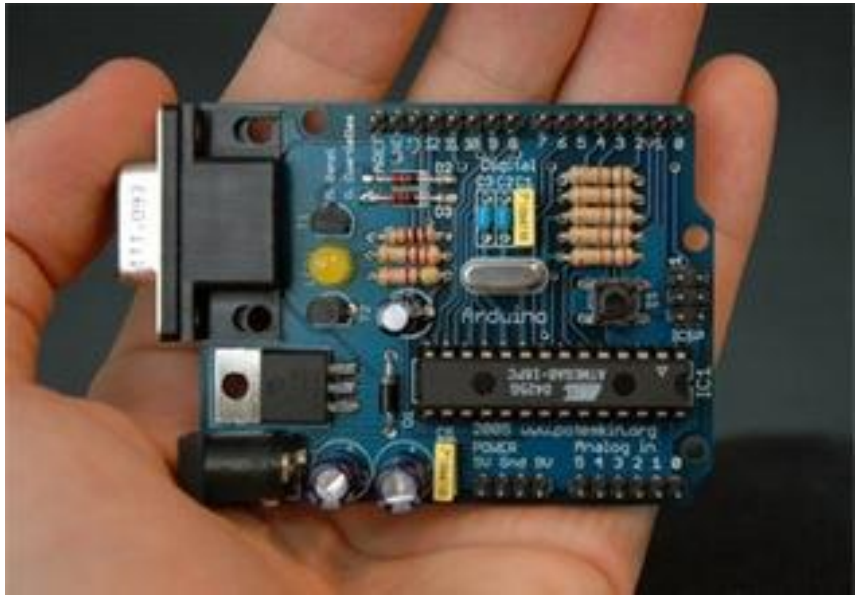
- Non trivial process even for simple projects
- Some challenges are inherent to the multidisciplinary nature of the field and it's good for students to struggle with them (e.g. sensors, actuators, power, design)
- Some are not (e.g. debugging C code, drivers issues, hardware limitations, unreadable manuals, high costs)

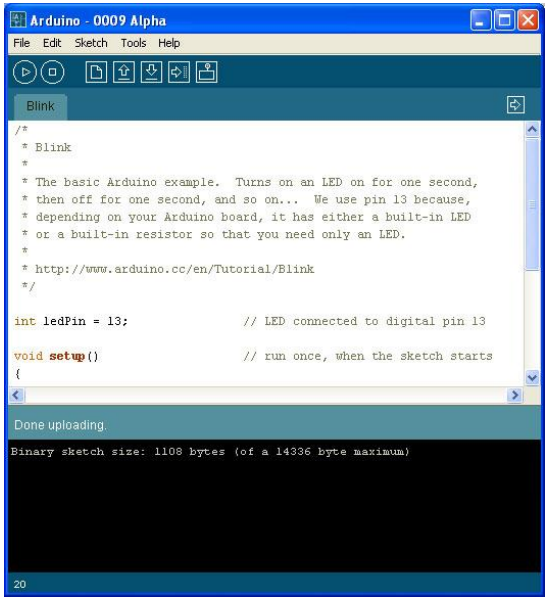
Outline

- A few challenges with mechatronics projects
- Arduino introduction, and MathWorks packages
- Analog and Digital IO, blink challenge example
- Servo, DC, and Stepper Motors, Encoders
- Simulink blocks, more details, and useful links

Arduino

- Arduino is an open-source microcontroller board, with an associated development environment.





Specifications (Arduino Uno)

- ATmega328 microcontroller
- 16 MHz, 32 KB FLASH, 2KB SRAM, 1K EEPROM
- 19 DIO pins (6 can be 8-bits 500Hz PWM outputs)
- 6 analog inputs (10 bits over 0-5V range, 15kSPS)
- 5V operating voltage, 40 mA DC Current per IO Pin
- I2C (TWI) fully supported and SPI partially supported
- Atmega16U2 acting as a USB-to-serial converter
- The OS sees it as a virtual serial port
- Power jack and optional 9V power supplier

Shields

- Shields are boards to be mounted on top of the Arduino
- They extend its functionality to control different devices, acquire data, and so on …

- Examples:
  - Motor/Stepper/Servo Shields (Motor Control)
  - Multichannel Analog and Digital IO Shields
  - Prototyping Shields
  - Ethernet and Wireless communication Shields
  - Wave Shields (Audio)
  - GPS Logging and Accelerometer Shields
  - Relay Control Shields

What is Arduino good for ?

- Projects requiring Analog and Digital IO
- Mechatronics Projects using Servo, DC or Stepper Motors
- Projects with volume/size and/or budget constraints
- Projects requiring some amount of flexibility and adaptability (i.e. changing code and functions on the fly)

What is Arduino good for ?

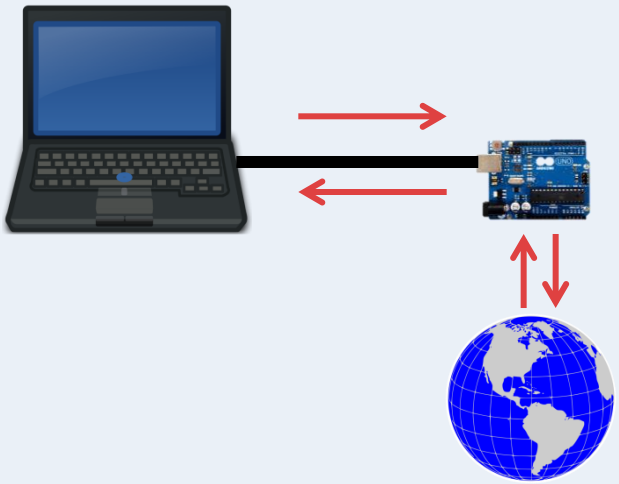
- Basically any Mechatronics project requiring sensing and acting, provided that computational requirements are not too high (e.g. can't do image processing with it)
- Ideal for undergraduate/graduate Mechatronics Labs and Projects
- There is a very large community of people using it for all kind of projects, and a very lively forum where it is possible to get timely support

Outline

- A few challenges with mechatronics projects
- Arduino introduction, and MathWorks packages
- Analog and Digital IO, blink challenge example
- Servo, DC, and Stepper Motors, Encoders
- Simulink blocks, more details, and useful links

Same board … different approaches

Communicate with the board from MATLAB (or Simulink) via USB



Tethered Approach

Develop a model and upload it on the board

Embedded Approach

MathWorks packages

- Arduino IO Package
  - Requires: MATLAB, Optional: Simulink
  - Approach: Tethered mode, Main Use: IO from MATLAB.

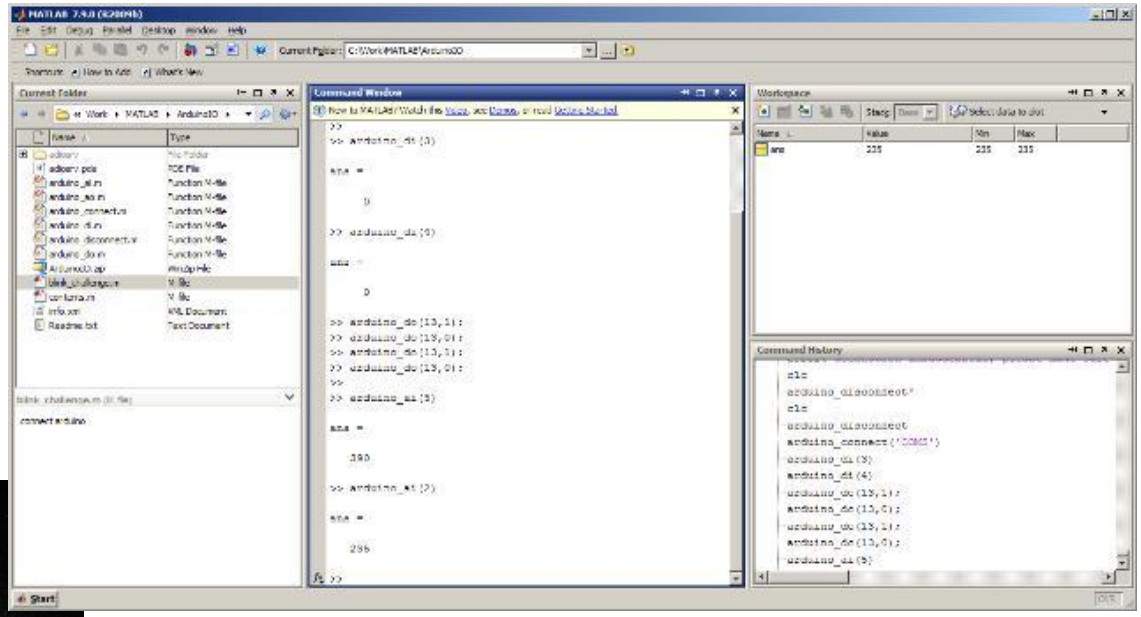
  This is the package explained in this presentation.

- Simulink Support Package for Arduino
  - Requires: MATLAB and Simulink
  - Approach: Embedded mode

- Embedded Coder Support Package for Arduino
  - Requires: MATLAB, Simulink, MATLAB Coder, Simulink Coder, Embedded coder, Approach: Embedded mode

Arduino IO package: concept

Arduino answers to MATLAB via USB



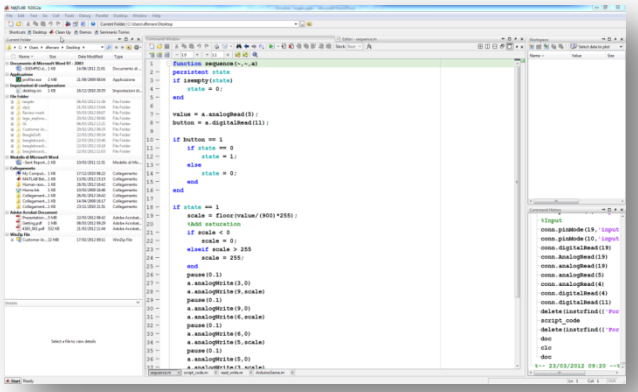
MATLAB sends a command or request to the Arduino board, via USB

The Arduino server sketch

- The server sketch is a Finite State Machine (written in C++) that continuously runs on the Arduino.

Request

Answer



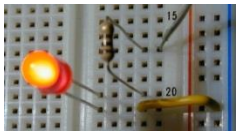
Arduino Side

PC Side (MATLAB)

- It **listens** for commands arriving from the serial port, **executes** the commands, and, if needed, **returns** an answer.

Which sketch do I need to upload ?

- There are 5 different server sketches in order of increasing complexity:



  - adio.pde : analog and digital IO & basic serial

  - adioe.pde : adio.pde + encoders support

  - **adioes.pde : adioe.pde + servo support**
  - motor_v1.pde : adioes.pde + Adafruit Motor Shield v1
  - motor_v2.pde : adioes.pde + Adafruit Motor Shield v2

Using MATLAB vs. the Arduino IDE

- MATLAB is more interactive, therefore results from Digital/Analog I/O instructions can be seen immediately without needing to program – compile – upload – execute each time. It is a good idea for algorithm prototyping

- MATLAB code is generally more compact and easier to understand than C (higher-abstraction data types, vectorization, no need for initialization/allocation, less lines of code) which means:
  a) MATLAB scales better with project complexity
  b) People get the job done faster in MATLAB

Using MATLAB vs. the Arduino IDE

- For wider-breadth projects (that might include data analysis, signal processing, calculations, simulation, statistics, control design …) MATLAB is better suited
- Engineering Departments typically need to introduce MATLAB during the first years, so this package will allow professors to keep the same environment and students to practice more MATLAB
- People might already be more familiar with MATLAB than with C, both in industry and university

Outline

- A few challenges with mechatronics projects
- Arduino introduction, and MathWorks packages
- Analog and Digital IO, blink challenge example
- Servo, DC, and Stepper Motors, Encoders
- Simulink blocks, more details, and useful links

Connect

- Use the command `a=arduino('port'),` with the right COM port as a string input argument, to connect MATLAB with the board and create an Arduino object in the workspace:

  Examples:

```
>> a=arduino('COM5');
>> a=arduino('/dev/ttyS101');
```

  Of course the board must be plugged in, with the drivers installed, and one of the supplied sketches needs to be running on it before the connection.

Assign pin mode (input/output)

- Use the command `pinMode(a,pin,str)` to get or set the mode of a specified pin:

- Examples:

```
>> pinMode(a,11,'output')
>> pinMode(a,10,'input')
>> a.pinMode(10,'input')
>> val=pinMode(a,10)
>> pinMode(a,5)
>> pinMode(a);
```

Digital read (digital input)

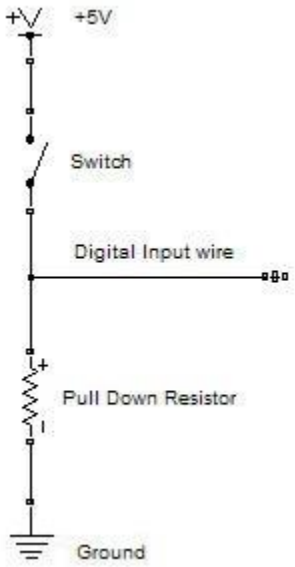
- Use the command `digitalRead(a,pin)` to read the digital status of a pin:

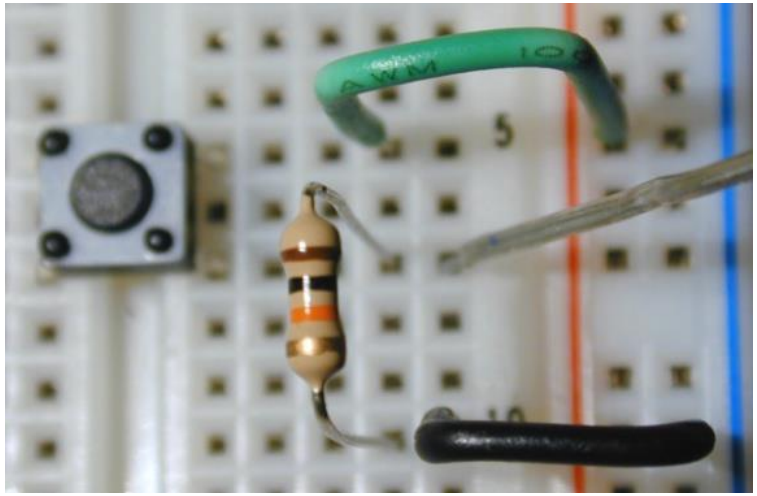
  Examples:

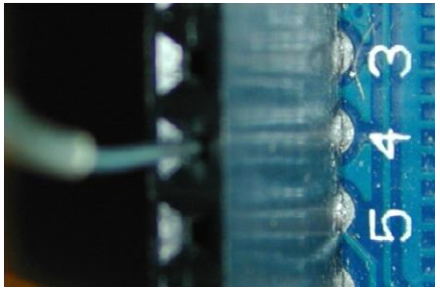
```
>> val=digitalRead(a,4)
>> val=a.digitalRead(4)
```

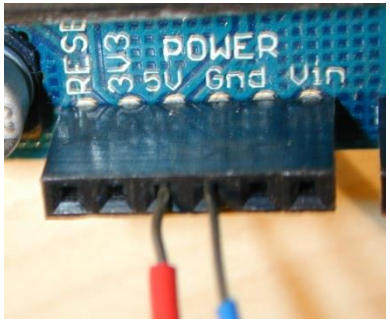
  This returns the value (0 or 1) read from the digital pin number 4

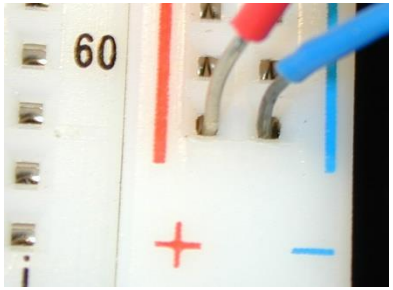
Digital read example











MATLAB Command Window

```
>> digitalRead(a,4)

ans =

     0

>> digitalRead(a,4)

ans =

     1
```

Command executed while button is released

Command executed while button is pressed

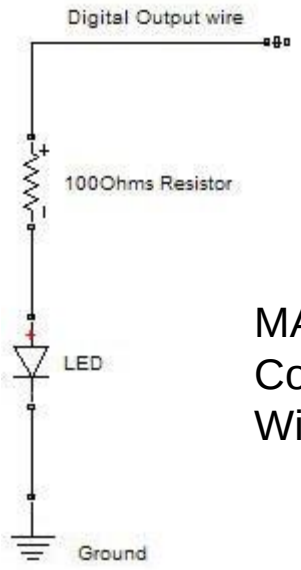
Digital write (digital output)

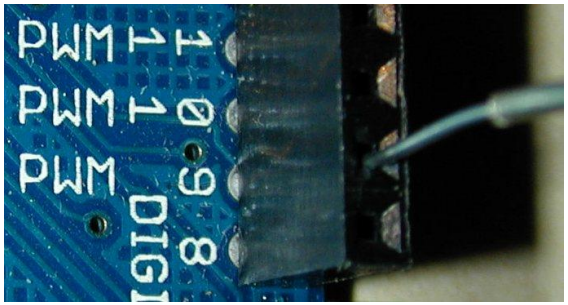
- Use the command `digitalWrite(a,pin,val)` with the pin as first argument and the value (0 or 1) as second argument:

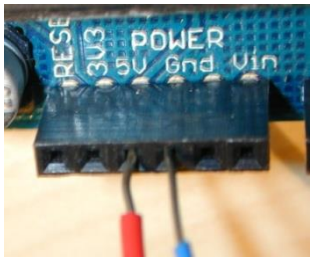
- Examples:

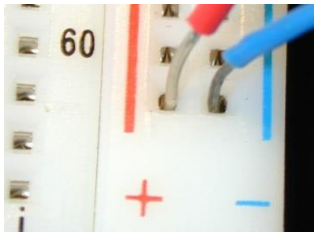
```
digitalWrite(a,13,1); % sets pin #13 high
a.digitalWrite(13,1); % sets pin #13 high
digitalWrite(a,13,0); % sets pin #13 low
```

Digital write example





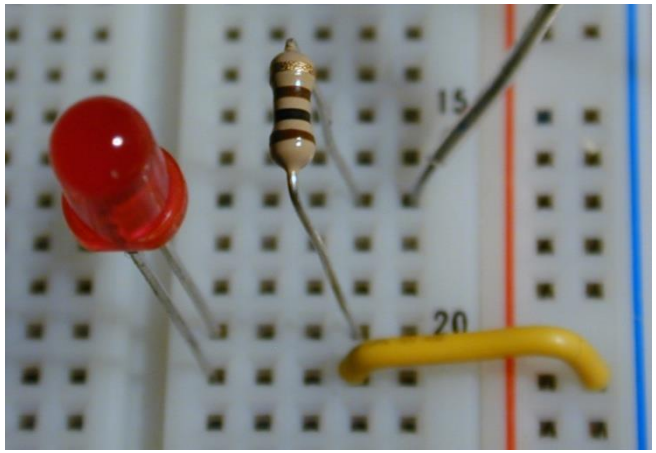




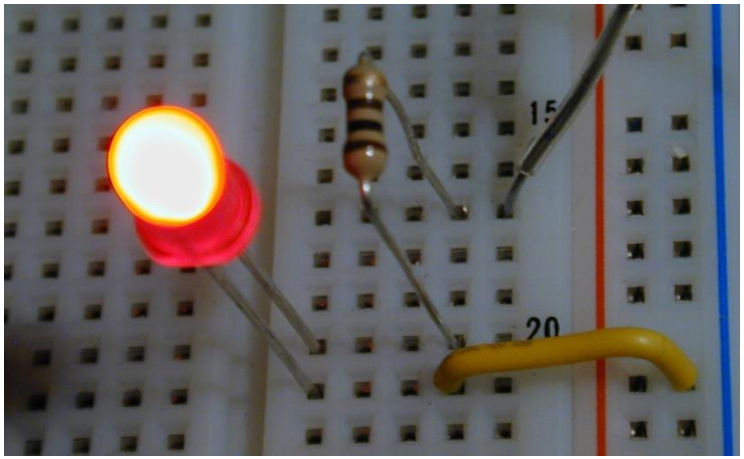
MATLAB Command Window

```
>> digitalWrite(a,9,1)
>> digitalWrite(a,9,0)
```

Led Off



Led On



Analog read (analog input)

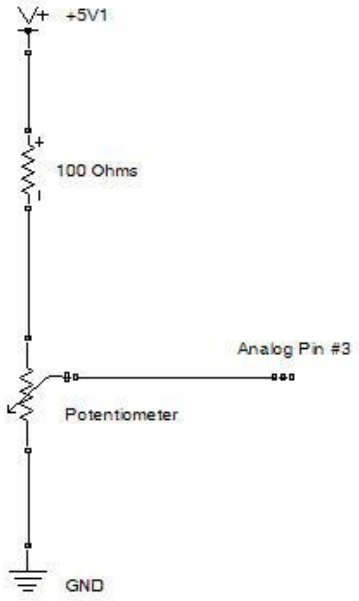
- Use the command `val=analogRead(a,pin)` with the pin as an integer argument:

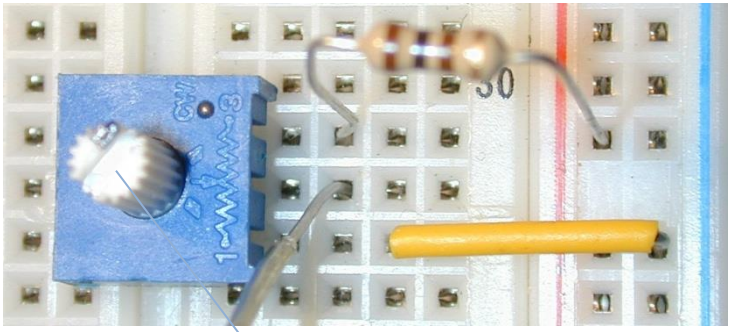
- Examples:

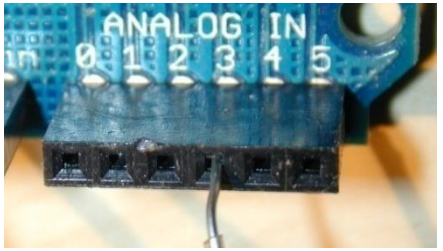
```
val=analogRead(a,0); % reads analog pin #0
val=a.analogRead(5); % reads analog pin #5
```

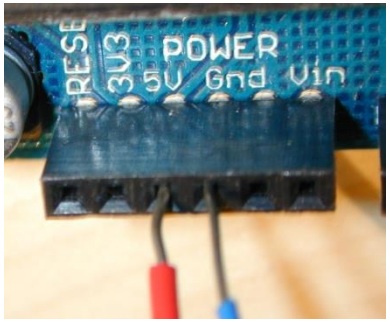
- The returned argument ranges from 0 to 1023
- Note that the 6 analog input pins (0 to 5) can be also addressed as digital pins 14 to 19 and are located on the bottom right corner of the board

Analog read example



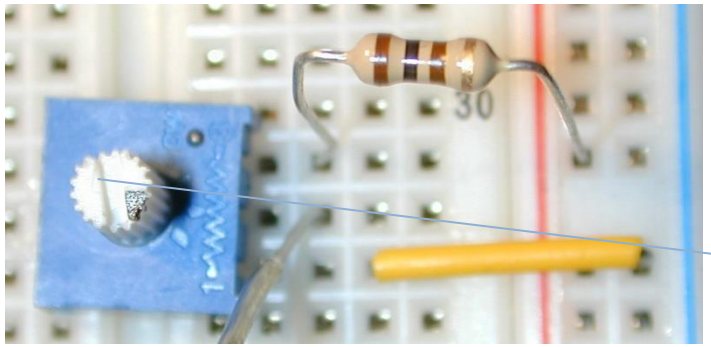


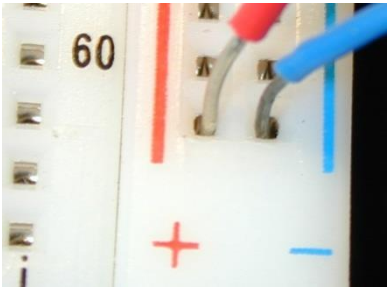




MATLAB Command Window

```
>> analogRead(a,3)

ans =

     285

>> analogRead(a,3)

ans =

     855
```





Analog (PWM) write (analog output)

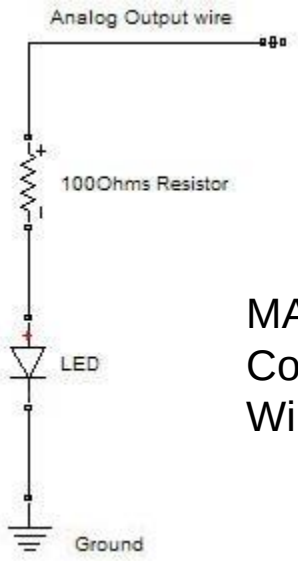
- Use the command `analogWrite(a,pin,val)` with the pin as first argument and the value (0 to 255) as second argument:

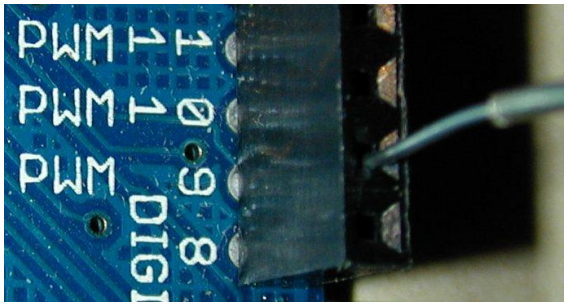
- Examples:

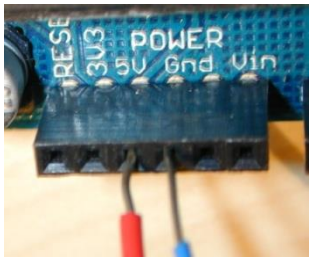
```
analogWrite(a,11,90); % sets pin #11 to 90
a.analogWrite(11,90); % sets pin #11 to 90
analogWrite(a,3,10); % sets pin #3 to 10
```

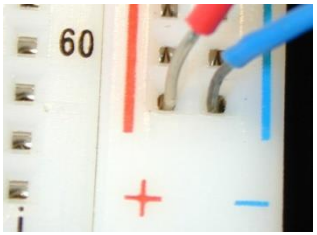
- On Uno boards allowed pins are 3, 5, 6, 9, 10 and 11. The PWM frequency is approx. 980Hz on pins 5 and 6 and approx. 480Hz on the other pins.

Analog write example





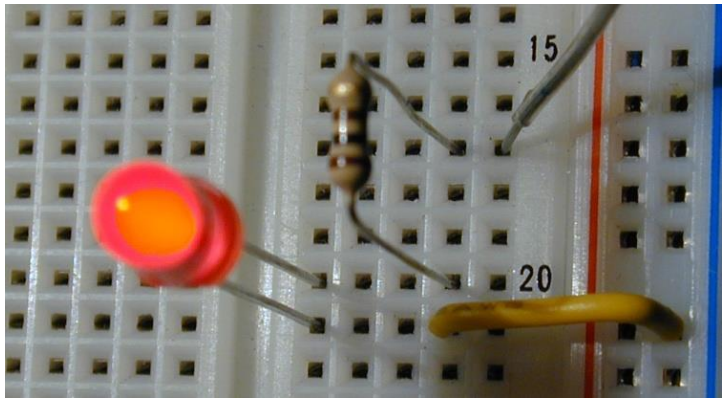




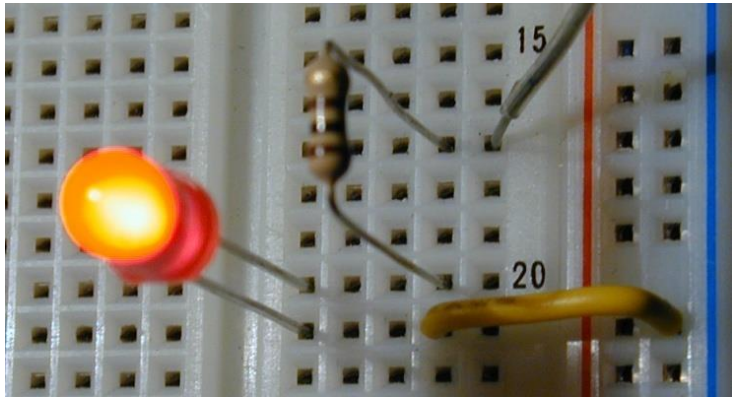
MATLAB Command Window

```
>> analogWrite(a,9,10)
>> analogWrite(a,9,50)
```

Led On (4%)



Led On (20%)



Disconnect

- Use the command `delete(a)` to disconnect the MATLAB session from the Arduino board:

  Examples:

```
>> delete(a);
>> a.delete;
```

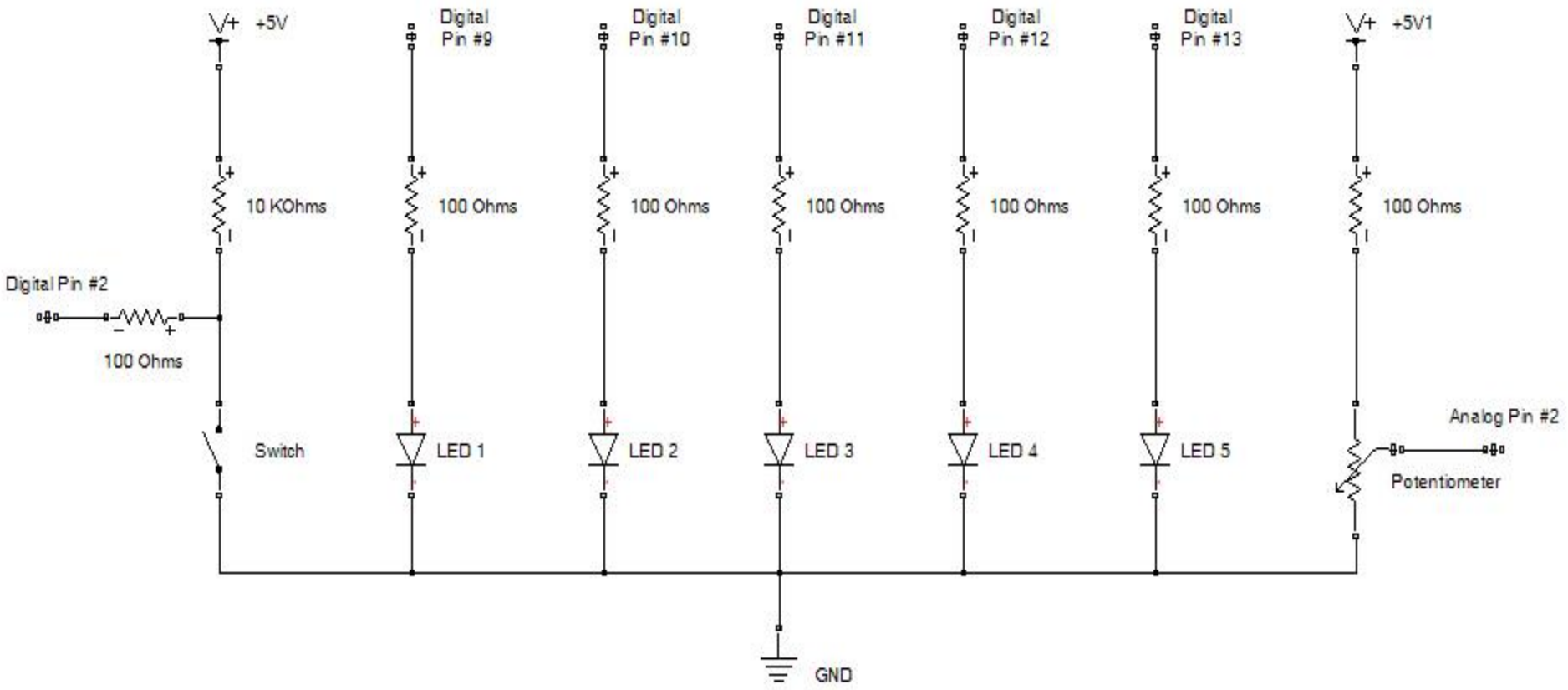
- This renders the serial port available for other sessions or the IDE environment

Example : the blink challenge project

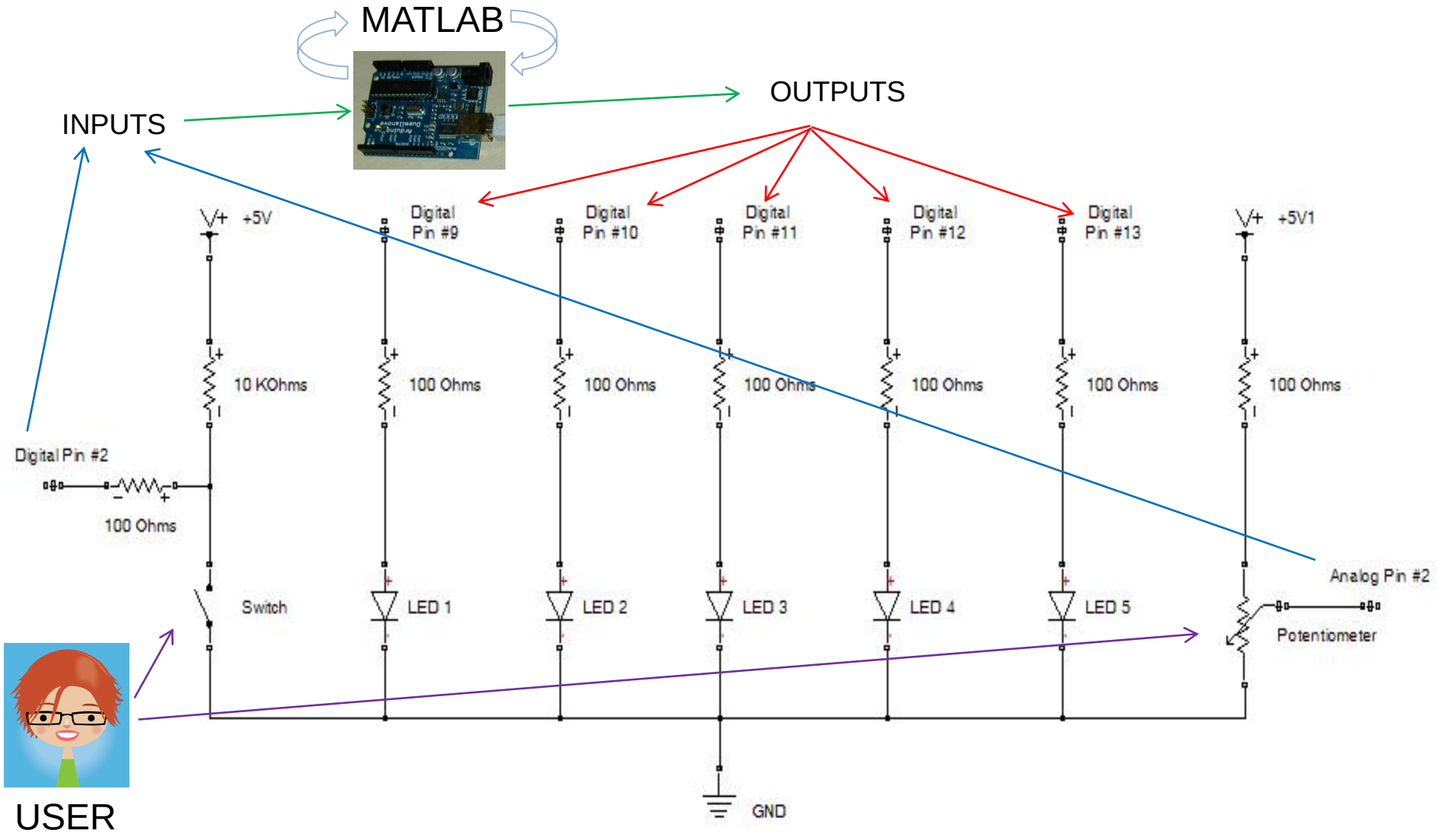
- This challenge is described in the last part of the Ladyada Arduino tutorial, http://www.ladyada.net/learn/arduino/ and it consists in designing a circuit with 5 LEDs and 4 modes (user switches mode by pushing a button):

1. All LEDs Off
2. All LEDs On
3. LEDs blinking simultaneously with variable frequency regulated by a potentiometer
4. LEDs blinking one after the other (wave like) with variable speed regulated by a potentiometer

Schematic


+5V
Digital Pin #9
Digital Pin #10
Digital Pin #11
Digital Pin #12
Digital Pin #13
+5V1
10 KOhms
100 Ohms
100 Ohms
100 Ohms
100 Ohms
100 Ohms
100 Ohms
Digital Pin #2
100 Ohms
Switch
LED 1
LED 2
LED 3
LED 4
LED 5
Analog Pin #2
Potentiometer
GND

Information flow



MATLAB code

```
% initialize pins
disp('Initializing Pins ...');

% sets digital input pins
a.pinMode(2, 'INPUT');
a.pinMode(3, 'INPUT');
a.pinMode(4, 'INPUT');
a.pinMode(7, 'INPUT');
a.pinMode(8, 'INPUT');

% sets digital and analog (pwm) output pins
a.pinMode(5, 'OUTPUT'); % pwm available here
a.pinMode(6, 'OUTPUT'); % pwm available here
a.pinMode(9, 'OUTPUT'); % pwm available here
a.pinMode(10,'OUTPUT'); % pwm available here
a.pinMode(11,'OUTPUT'); % pwm available here
a.pinMode(12,'OUTPUT');
a.pinMode(13,'OUTPUT');

% button pin and analog pin
bPin=2;aPin=2;

% initialize state
state=0;

% get previous state
prev=a.digitalRead(bPin);

% start loop
disp('Starting main loop, push button to change state ...');

% loop for 1 minute
tic
while toc/60 < 1

  % read analog input
  ain=a.analogRead(aPin);
  v=100*ain/1024;

  % read current button value
  % note that button has to be kept pressed a few seconds to make sure
  % the program reaches this point and changes the current button value
  curr=a.digitalRead(bPin);

  % button is being released, change state
  % delay corresponds to the "on" time of each led in state 3 (wave)
  if (curr==1 && prev==0),
     state=mod(state+1,4);
     disp(['state = ' num2str(state) ', delay = ' num2str(v/200)]);
  end
```

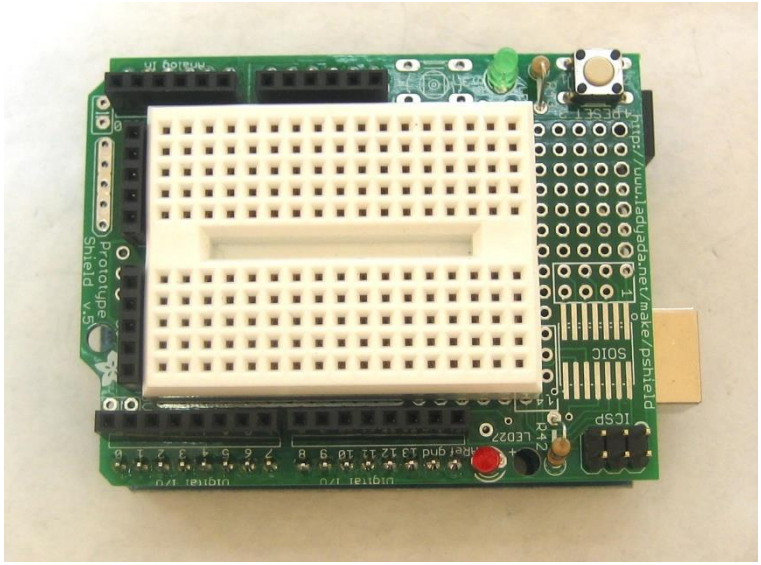
MATLAB code

```
% toggle state all on or off
   if (state<2),
      for i=9:13,
         a.digitalWrite(i,state);
      end
   end


% blink all leds with variable delay
   if (state==2),
      for j=0:1,
         % analog output pins
         for i=9:11,
            a.analogWrite(i,20*(i-8)*j);
         end
         % digital output only pins
         for i=12:13,
            a.digitalWrite(i,j);
         end
         pause((15*v*(1-j)+4*v*j)/1000);
      end
   end
```

```
% wave
   if (state==3),
      for i=4:8,
         a.digitalWrite(9+mod(i,5),0);
         a.digitalWrite(9+mod(i+1,5),1);
         pause(v/200);
      end
      a.digitalWrite(13,0);
   end


% update state
   prev=curr;

end



% turn everything off
for i=9:13, a.digitalWrite(i,0); end
```

Prototyping shield

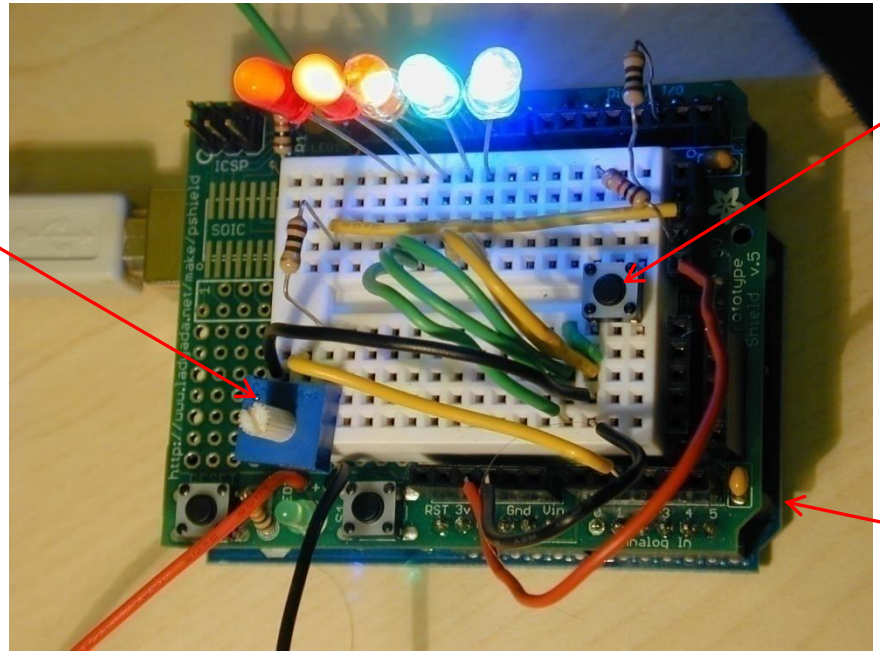
- The schematic was implemented using the prototyping shield:



- This shield allows for an easy prototyping of projects based on the Arduino board

Implementation

Potentiometer to regulate on/off LEDs delay



Button to change mode

Arduino Board underneath

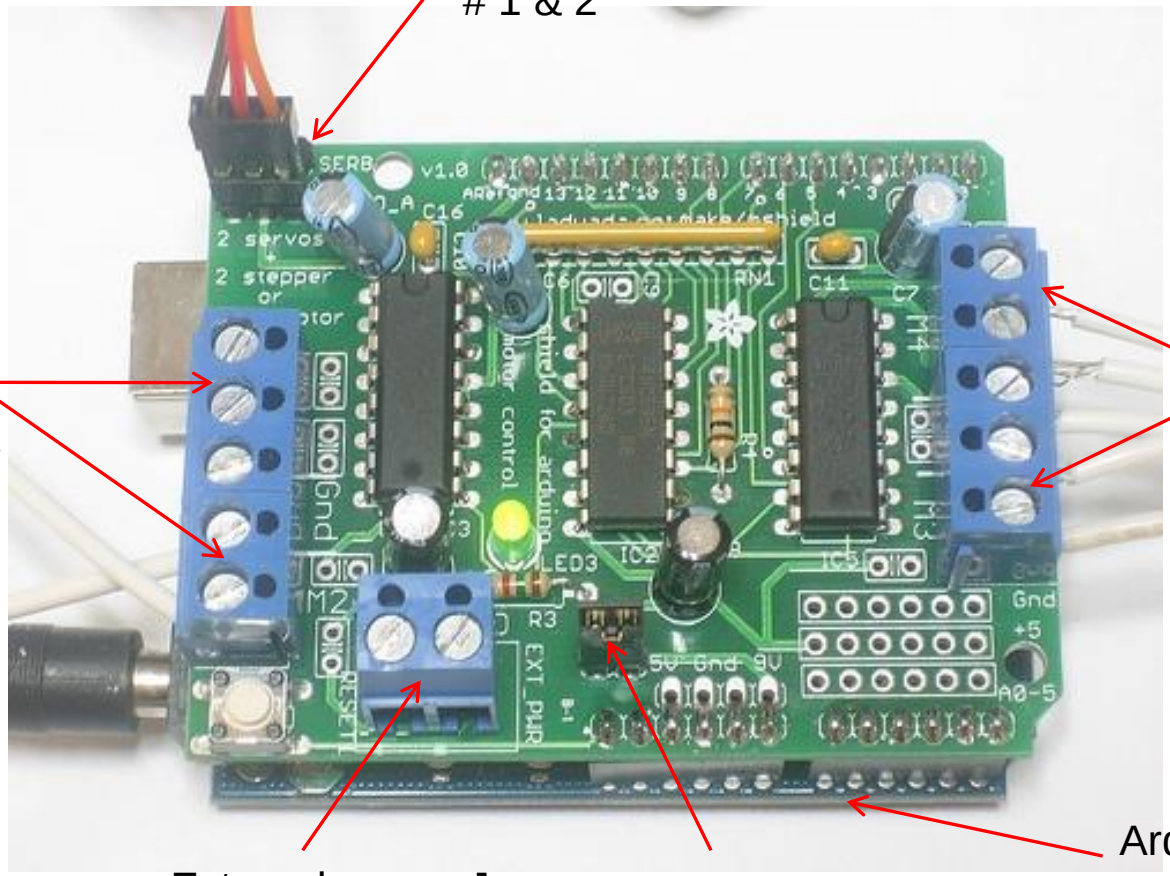
Outline

- A few challenges with mechatronics projects
- Arduino introduction, and MathWorks packages
- Analog and Digital IO, blink challenge example
- DC and Stepper motors, Servos, Encoders
- Simulink blocks, more details, and useful links

Motors and motor shields, quick facts

- Reversing the rotation direction of a Direct Current (DC) motor requires an H-Bridge circuit.
- More powerful DC and Stepper motors often require an external power source.
- Motor shields typically provide both an H-Bridge circuit and connectors for an external power source.
- The “official” Arduino Motor Shield, and the Adafruit (AF) Motor Shield (Version 1 and Version 2) are very common.

Example: Adafruit motor shield v1



Example: Adafruit motor shield v1 specs

- **2 connections for 5V 'hobby' servos** connected to the Arduino's high-resolution dedicated timer
- **Up to 4 bi-directional DC** motors with individual 8-bit speed selection
- **Up to 2 stepper motors** (unipolar or bipolar) with single coil, double coil, interleaved or micro-stepping.
- **4 H-Bridges:** L293D chipset provides 0.6A per bridge (1.2A peak) with thermal shutdown protection, 4.5V to 36V

Motor shields, more quick facts

- The Arduino IO package has specific commands for DC and Stepper motors with Adafruit Motor shields
- This requires using specific Adafruit Motor libraries and uploading on the board either the Motor_v1.pde or Motor_v2.pde sketch (see the readme.txt file).

- The official Arduino Motor Shield only handles 2 DC motors (no stepper) and it does not rely on any custom library.
- Motors can be operated using just 6 digital pins (3, 8, 9, 11, 12, 13), there are no specific commands or libraries, and therefore the adio.pde sketch is all that is needed to operate this shield with the Arduino IO package.

DC motor speed (Adafruit motor shield)

- Use the command `val=motorSpeed(a,num,spd);` to get or set the speed of a DC motor.
- The second argument, `num`, is the number of the motor (1 to 4) the third argument is the speed (0 to 255)

- Examples:

```
motorSpeed(a,4,200);
val=motorSpeed(a,1);
motorSpeed(a,3);
a.motorSpeed;
```

- Note: nothing moves unless we issue a “motorRun” command …

DC motor run (Adafruit motor shield)

- Use the command `motorRun(a,num,dir);` to run a given DC motor.
- The second argument, `num`, is the number of the motor (1 to 4) the third argument is a string that can be either 'forward', 'backward', 'release'

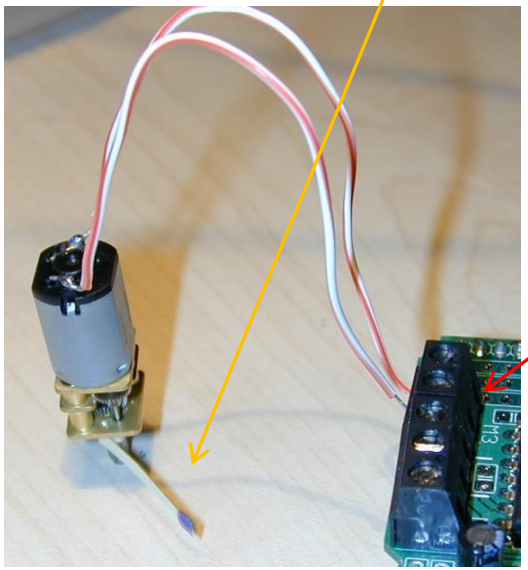
- Examples:

```
motorRun(a,1,'forward');
motorRun(a,3,'backward');
a.motorRun(3,'backward');
motorRun(a,1,'release');
```

DC motor (Adafruit motor shield): example

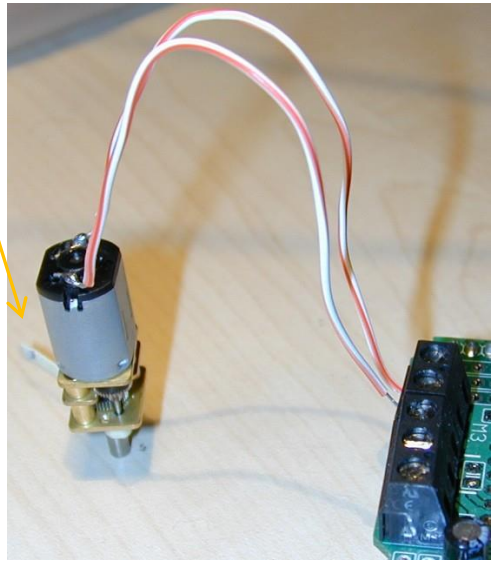
MATLAB Command Window:

```
>> motorSpeed(a,3,100)

>> motorRun(a,3,'forward')
```



Motor #3

(running)



Stepper motor speed (Adafruit motor shield)

- Use the command `val=stepperSpeed(a,num,spd);` to get or set the speed of a stepper motor.
- The second argument, `num`, is the number of the motor (1 to 4) the third argument is the speed in RPM (0 to 255)

- Examples:

```
stepperSpeed(a,2,50)
val=stepperSpeed(a,1);
a.stepperSpeed(3);
stepperSpeed(a);
```

- Again, nothing moves unless we issue a “stepperStep” command.

Stepper motor step (Adafruit motor shield)

- Use the command `stepperStep(a,num,dir,sty,steps);` to advance a stepper motor of a certain number of steps.
- Where `num` is the number of the stepper (1 or 2), `dir` can be either 'forward', 'backward', 'release', `sty` is the style of motion and can be 'single', 'double', 'interleave', 'microstep', `steps` is the number of steps (0 to 255)

- Examples:

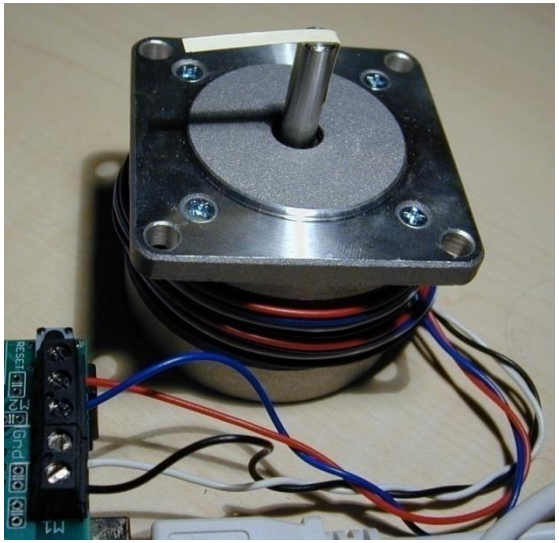
```
stepperStep(a,1,'forward','interleave',50);
a.stepperStep(1,'forward','microstep',200);
stepperStep(a,1,'backward','double',200);
stepperStep(a,1,'release');
```

Stepper motor (Adafruit motor shield): example

MATLAB Command Window:

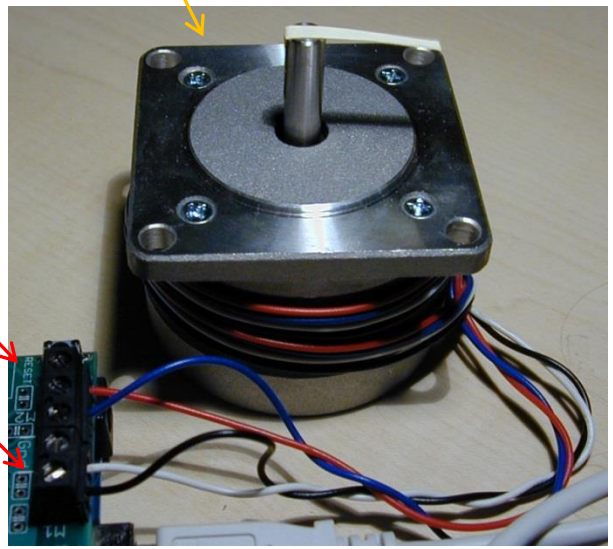
```
>> stepperSpeed(a,1,50)

>> stepperStep(a,1,'backward','microstep',100)
```

Stepper #1



Coil #1

Coil #2



Servo motors, quick facts

- Servo motors come already packaged with a rotation sensor (usually a potentiometer) and a control circuit.
- The reference position is communicated to the control circuit via a PWM signal.
- The Arduino has a native servo library which allows a servo to be connected to any PWM pin (specifically the central red wire goes to +5V, the black or brown one goes to ground, and the white or orange one must be connected to a PWM pin).

Servo motors, more quick facts

- Servos tend to draw a lot of power so it is better to power them using a dedicated power source. Powering even small servos from the Arduino USB connection might cause resets and connection losses.

- While motor shields are not required to use servos, they make it easier to do so as they typically have dedicated connectors for both servos and external power sources.

- You can operate servos with either adioes.pde, Motor_v1, or Motor_v2 sketches included in the Arduino IO package.

Servo status (attached/detached)

- Use the command `val=servoStatus(a,pin)` to get the status of the servo attached to a certain PWM pin, which can be either:
  - attached (ready for read or write)
  - detached

- Examples:

```
val=servoStatus(a,9);
servoStatus(a,10);
a.servoStatus(11);
servoStatus(a);
```

Servo attach

- Use the command `servoAttach(a,pin)` to attach a servo to the corresponding PWM pin

- Examples:

```
servoAttach(a,10); % attach servo to pin 10
a.servoAttach(10); % attach servo to pin 10
servoAttach(a,9); % attach servo to pin 9
```

- Note: on the AF Motor Shield v1 there are 2 servo connectors, the outermost one uses pin #10, the innermost one uses pin #9.

Servo read

- Use the command `val=servoRead(a,pin)` to read the angle from the servo connected to a given PWM pin
- The returned value is the angle in degrees, typically from 0 to 180.

- Examples:

```
val=servoRead(a,10); % read angle servo #10
val=a.servoRead(10); % read angle servo #10
val=servoRead(a,9); % read angle servo #9
```

Servo write

- Use the command `servoWrite(a,pin,angle)` to rotate a servo of a given angle.
- The first argument is the number of the servo, the second is the angle.

- Examples:

```
servoWrite(a,10,45); % rotate servo 10 to 45°
a.servoWrite(10,70); % rotate servo 10 to 70°
```

Servo detach

- Use the command `servoDetach(a,pin)` to detach a servo from the corresponding PWM pin.

- Examples:

```
servoDetach(a,10); % detach servo #10
a.servoDetach(10); % detach servo #10
servoDetach(a,9); % detach servo #9
```

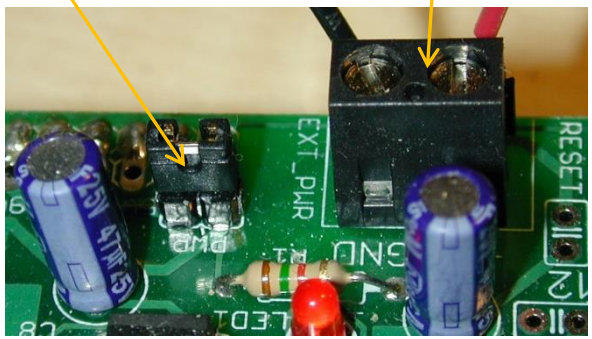
Servo example (with Adafruit motor shield v1)

MATLAB Command Window:

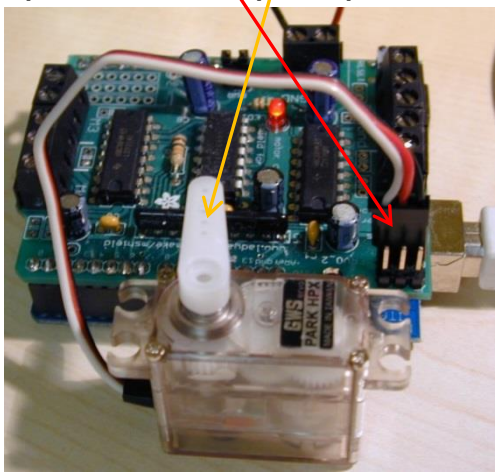
```
>> servoAttach(a,9)

>> servoWrite(a,9,5)

>> servoWrite(a,9,180)

>> servoRead(a,9)

ans =

   180
```

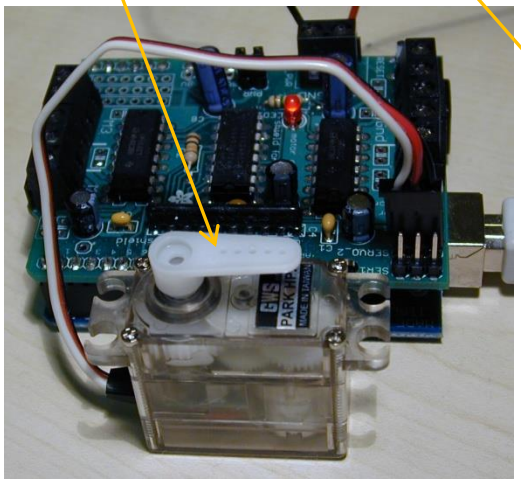
Power Jumper on (suggested)

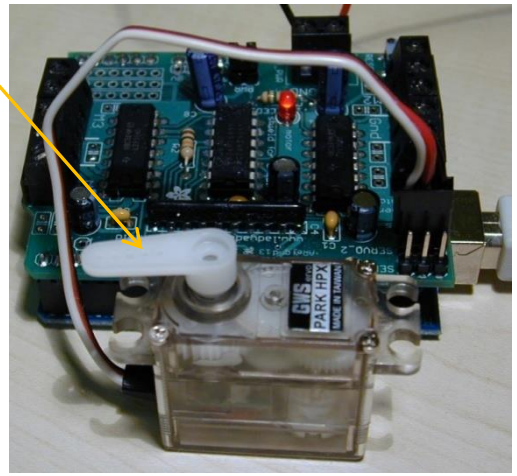
External Power: 6V Lantern Battery



Servo #2 (attached to pin 9)







Encoder quick facts

- Incremental rotary encoders are used to accurately measure rotations (angles and angular speeds) .

- They have 3 terminals. The central (common) one should be connected to ground, while the other two terminals to digital input pins to which interrupt service routines (ISRs) can be attached, (the ISR updates a variable holding the current encoder position as the encoder is rotated, the user can later read this variable from MATLAB).

- Any Arduino IO sketch except adio.pde can be used to work with encoders.

Encoder attach

- Use the command `encoderAttach(a,enc,pinA,pinB)` to attach an encoder to the interrupt pins `pinA` and `pinB`, `enc` is the encoder number (0, 1 or 2).

- Example:

```
encoderAttach(a,0,2,3); % attaches encoder
#0 on pins 2 and 3
```

Encoder status (attached/detached)

- Use the command `val=encoderStatus(a,enc)` to get the status of the encoder `enc`, which can be either:
  - attached (ready for read or reset)
  - detached

- Examples:

```
encoderDetach(a,0); % detach encoder #0
a.encoderDetach(1); % detach encoder #1
```

Encoder read

- Use the command `val=encoderRead(a,enc)` to read the rotation from a given encoder (`enc`). The returned value is an integer from -32768 to +32767 (clockwise rotations are positive).

- Examples:

```
val=encoderRead(a,2); % reads encoder #2
val=a.encoderRead(0); % reads encoder #0
```

- Note, an `encoderDebounce(a,enc,delay)` function can be used, if necessary, to set a debounce delay (see readme.txt and documentation for more info).

Encoder reset

- Use the command `encoderReset(a,enc)` to reset (to zero) the variable holding the position of encoder `enc`.

- Examples:

```
encoderReset(a,1); % resets encoder #1
a.encoderReset(2); % resets encoder #2
```

Encoder detach

- Use the command `encoderDetach(a,enc)` to detach the encoder `enc` from the corresponding interrupt pins.

- Examples:

```
encoderDetach(a,0); % detach encoder #0
a.encoderDetach(1); % detach encoder #1
```

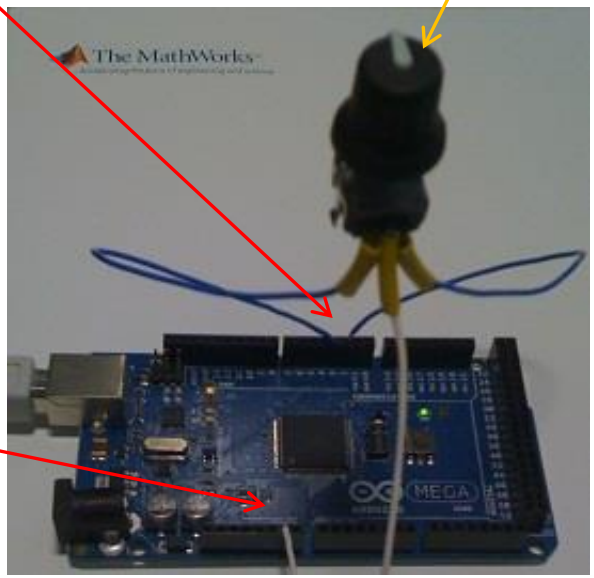
Encoder example

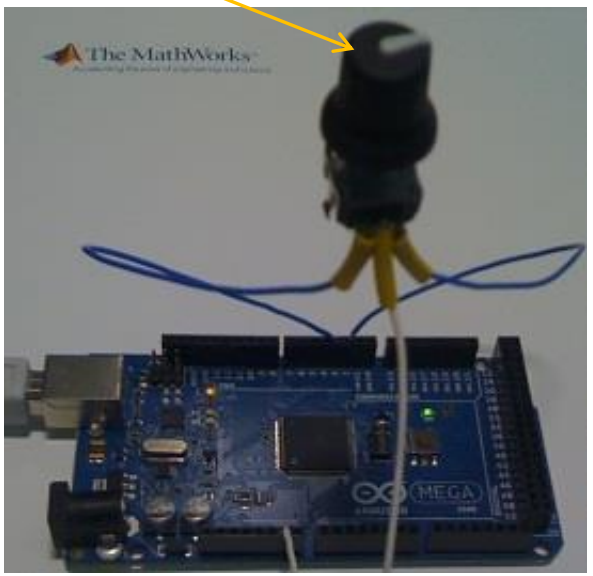
MATLAB Command Window:

```
>> encoderAttach(a,0,3,2)

>> encoderRead(a,0)

ans =

     0

>> encoderRead(a,0)

ans =

    44
```

Encoder terminals A and B attached to pins 3 and 2

Encoder terminal C (common) attached to ground

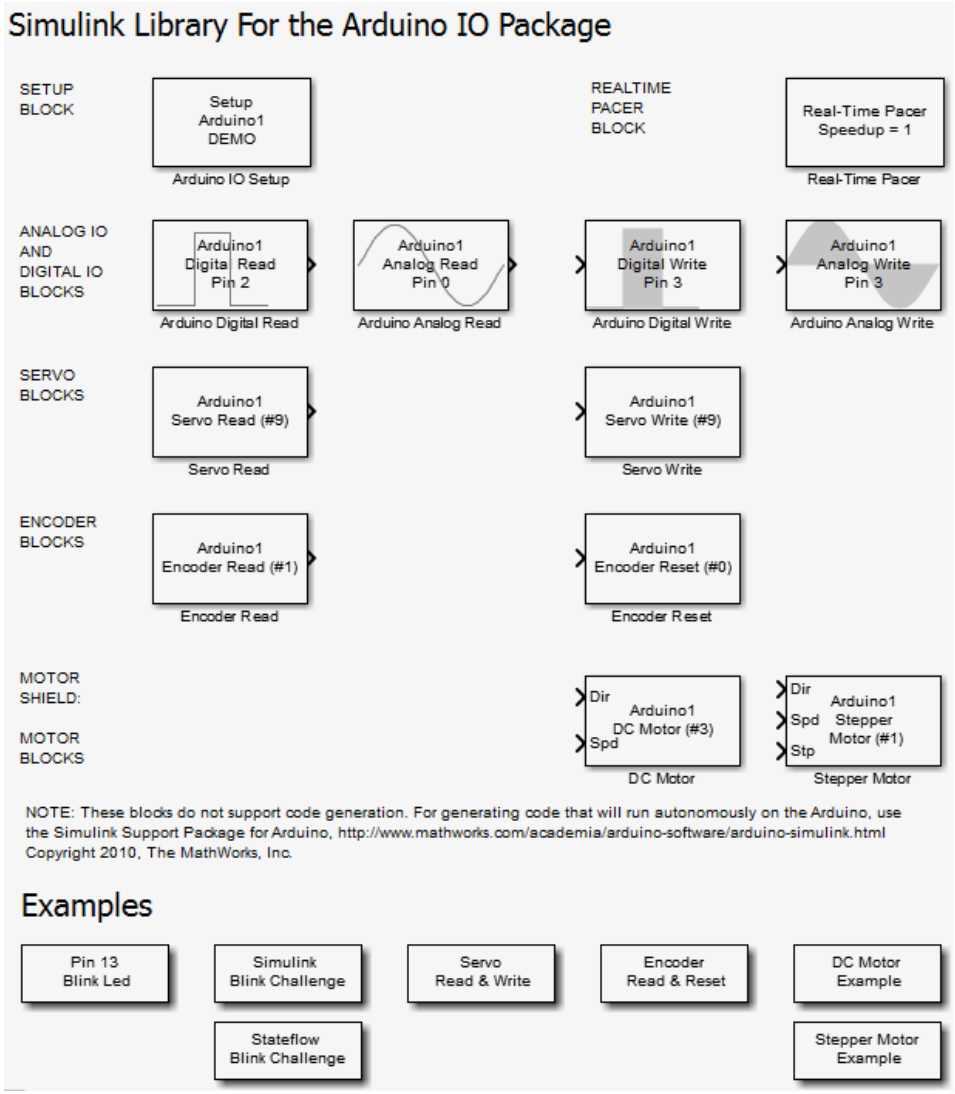




Outline

- A few challenges with mechatronics projects
- Arduino introduction, and MathWorks packages
- Analog and Digital IO, blink challenge example
- DC and Stepper motors, Servos, Encoders
- Simulink blocks, more details, and useful links

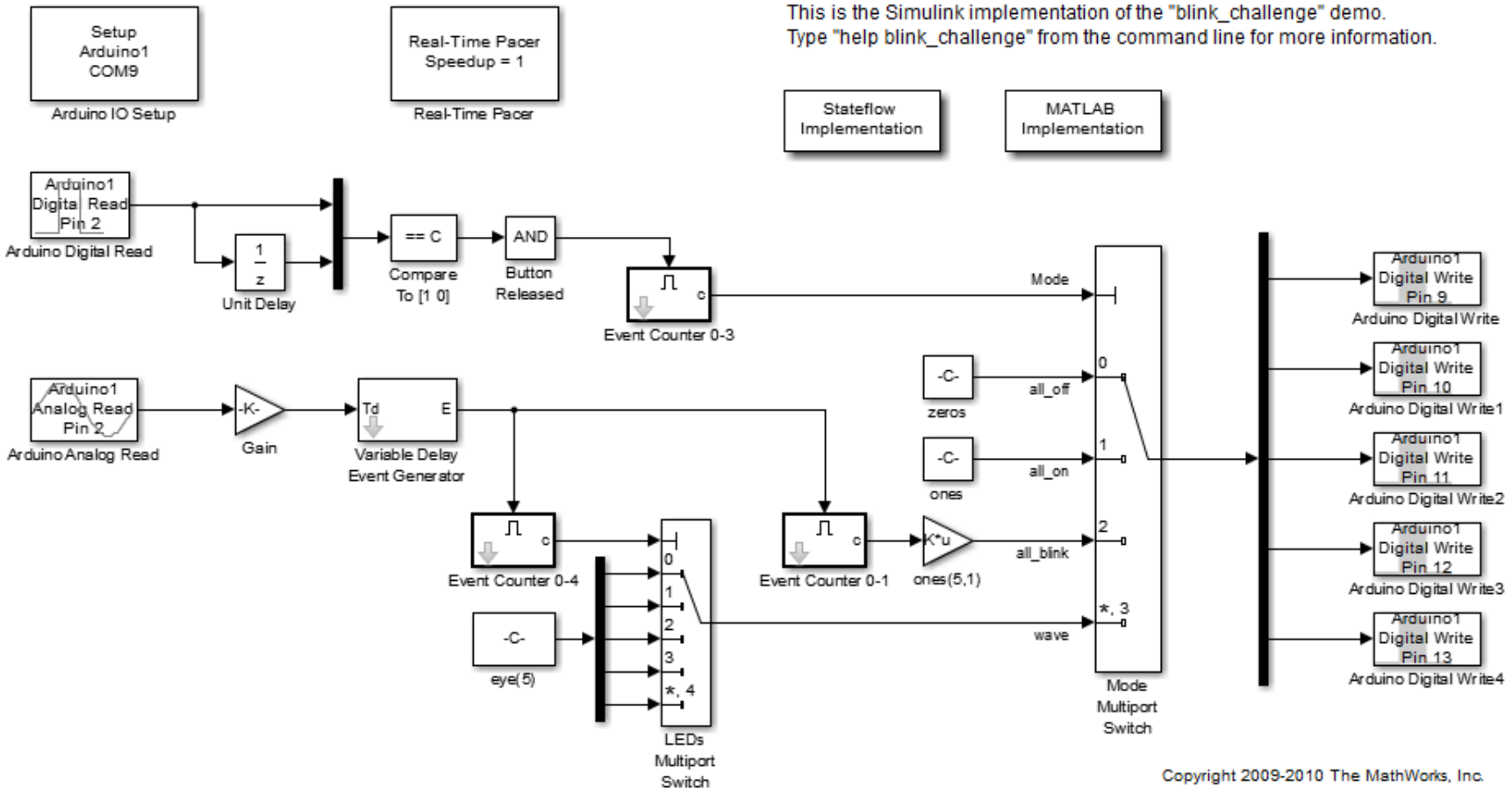
Arduino IO Simulink library



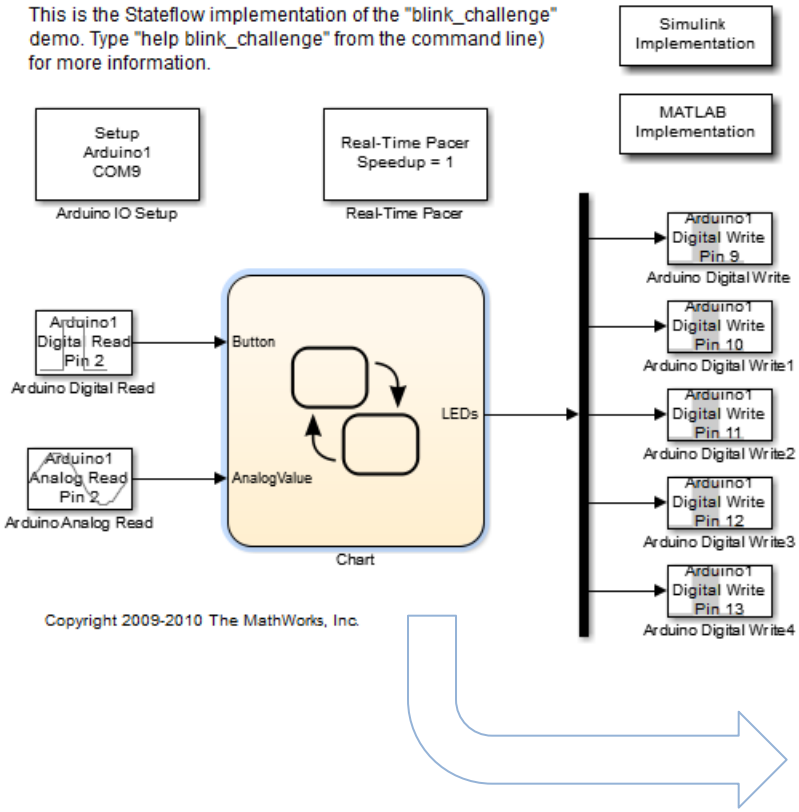
- 12 blocks:
- Setup and Pacer
- Analog and Digital IO
- Servos
- Encoders
- AF Motor Shield

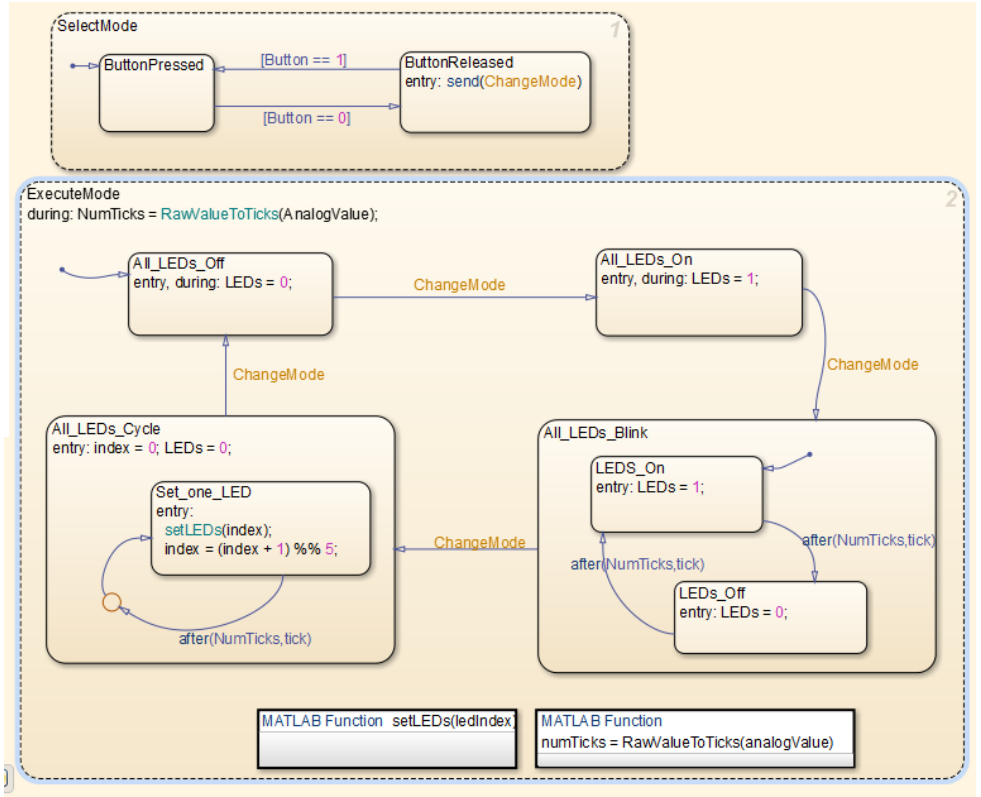
- Blocks call the corresponding Arduino IO MATLAB function each time they are executed.

Example: blink challenge using Simulink



Example: blink challenge using Stateflow





Outline

- A few challenges with mechatronics projects
- Arduino introduction, and MathWorks packages
- Analog and Digital IO, blink challenge example
- DC and Stepper motors, Servos, Encoders
- Simulink blocks, more details, and useful links

Server sketch: basic architecture

```
void loop() {
…
        if (Serial.available() >0) {
        val = Serial.read();

        switch (s) {
        …
        case 20:
                action (20,val)
                s (20,val)
        …
```

val = one-byte *input* from the serial port

s = *state* (initialized to -1)

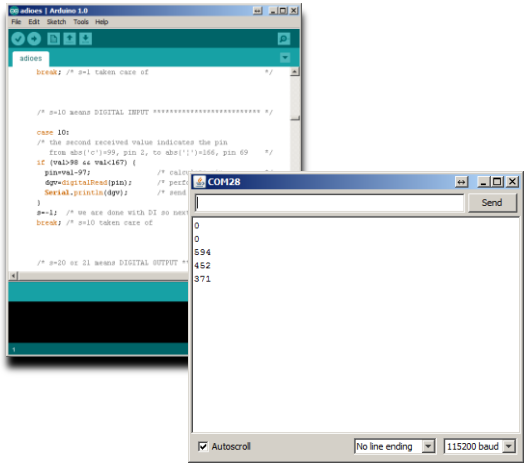
Calculate *next state* and perform an *action* based on both *current state* and *input*

Using the server with a serial monitor

- The server can be used without MATLAB with a serial terminal (or the IDE serial monitor) set on 115200 baud

Request

Answer



Arduino Side

PC Side: Serial Terminal
(*NOTE: Not MATLAB*)

Using the server with a serial monitor

- Examples:
  - 1c : reads digital pin #2 (c)
  - 2n1 : sets digital pin #13 (n) high
  - 4jz : analog output on pin #9 (j) to 122=ascii(z) over 255
  - 6j1 : attaches servo on pin #9
  - A1z : sets speed of motor #1 to 122 over 255 (122=ascii(z))
  - D1fsz : does 122 steps on motor #1 forward in single (s) mode
  - E0cd : attaches encoder #0 (0) on pins 2 (c) and 3 (d)
  - G0 : gets position of encoder #0
  - R2 : sets analog reference to EXTERNAL
- This is a good debugging strategy (e.g it allows inserting printf statements in the sketch).

Customizing Arduino IO: roundTrip

The roundTrip function sends a value to the Arduino, the board receives this value (see “case :400” in the code), and then sends it back to MATLAB:

Sends byte

Receives byte

and sends it back

```
>> roundTrip(a,42)

ans =

    42
```

Arduino Side

PC Side (MATLAB)

You can use this function as a *starting point to customize the server sketch* and introduce your own code.

Supported boards

- Arduino boards:
  - Duemilanove
  - Uno
  - Mega 2560
  - Nano
  - Due (only adio.pde and adioe.pde)
  - Other boards like Mini or Pro may work but have not been tested.
- Digilent ChipKit boards:
  - Uno32 (adio.pde only)
  - Max32 (adio.pde only)
  - Cerebot (adio.pde only)
- See the readme.txt file for more info

Useful links: buying an Arduino

- An extensive list of sites where Arduino can be bought: http://www.arduino.cc/en/Main/Buy
- In the US, Adafruit industries: http://www.adafruit.com/ provides a starter pack that includes pretty much everything that you need to get started with the board.
- Sparkfun: https://www.sparkfun.com/
- Robotshop: http://www.robotshop.com/

Useful links: buying motors

- RC Servos from Pololu: http://www.pololu.com/catalog/category/23
- DC Motors from Pololu: http://www.pololu.com/catalog/category/51
- Jameco for Stepper Motors and pretty much everything: http://www.jameco.com/
- List of Hobbyist and Surplus Stores: http://www.ladyada.net/library/procure/hobbyist.html

Useful links: getting started guides

- Official Arduino web site: http://arduino.cc/en/
- Knowledge base: http://www.freeduino.org/

- Official Getting Started guide: http://arduino.cc/en/Guide/HomePage
- The LadyAda Tutorial: http://www.ladyada.net/learn/arduino/

- Download the MATLAB Arduino IO Package: http://www.mathworks.com/matlabcentral/fileexchange/32374

Conclusions

- Arduino is an inexpensive open-source microcontroller board, well suited for a wide range of projects
- Arduino + ArduinoIO package + MATLAB = inexpensive and interactive Analog and Digital IO from the MATLAB command line
- Arduino + Motor Shield + ArduinoIO package + MATLAB = inexpensive and interactive Motor Control from the MATLAB command line